

RRT* Motion Planning with Control Barrier Function Safety Derived from Poisson Fields

Maciej Rózewicz

AGH University of Science and Technology, Kraków, Poland,
mrozewicz@agh.edu.pl

Abstract. This paper proposes a novel framework for safe motion planning that integrates Control Barrier Function (CBF) safety constraints with the Rapidly-Exploring Random Tree Star (RRT*) algorithm. The CBF is derived directly from the solution of a Poisson equation defined over the free workspace, referred to as a *Poisson Safety Function (PSF)*. The PSF encodes obstacle boundaries and safety gradients in a smooth, differentiable field that naturally satisfies Laplace-type spatial regularity. The proposed method, termed *Safe-CBF-RRT**, uses the PSF and its spatial derivatives to enforce pCBF-based safety conditions along sampled edges during tree expansion. Unlike pure safety-filter approaches such as FaSTrack, our method actively generates an inherently safe path while maintaining computational tractability. Simulation results demonstrate that the method enhances safety and convergence properties compared to standard RRT*.

1 Introduction

In modern robotics applications, ensuring safe navigation in complex environments is one of the major challenges. This is particularly critical in scenarios involving human-robot interaction, autonomous vehicles, and unstructured terrains. Traditional motion planning algorithms often focus on finding feasible paths without formal guarantees of safety. To achieve such provable safety, Control Barrier Functions (CBFs) have emerged as a powerful tool for enforcing safety constraints in control systems in last years [1].

In parallel, methods like **FaSTrack** [4] have provided formal guarantees for *safe trajectory tracking* by defining a Safe-to-Track Set ($\mathcal{S}$), leveraging techniques from optimal control and differential games. However, such methods are primarily safety filters for the *execution* phase and *do not inherently provide an efficient planner to generate* the safe reference trajectory itself, often suffering from high computational cost in high-dimensional state spaces.

Sampling-based motion planners such as RRT or RRT* are widely used for complex robotic systems due to their scalability and ability to handle nonconvex constraints [9], [7]. However, they rely primarily on geometric collision checking, offering no formal guarantees of safety. For this reason, integrating CBFs into sampling-based planners seems to be a promising direction to enhance safety while retaining the benefits of sampling-based methods.

A critical challenge in using CBFs for planning is the construction of a suitable safety function $h(x)$, which must be continuously differentiable and satisfy certain regularity conditions. Analytical definitions of $h(x)$ exist for simple geometries, and lately there have been attempts to get CBF as solution of Poisson equation [3], [2]. This approach will be used in this paper to generate a Poisson Safety Function (PSF) that encodes obstacle boundaries and provides smooth safety gradients. Such definition of $h(x)$ is particularly advantageous for complex environments where analytical forms are not feasible. Moreover, it seems to be naturally compatible with the RRT or RRT* framework. **Contributions:** In this the *Safe-CBF-RRT** framework, the following contributions are made:

- Integration of the PSF into a modified RRT* algorithm.
- a computationally efficient alternative to tracking-based safety methods like FaSTrack for generating inherently safe reference trajectories.
- Empirical results showing improved safety margins and convergence.

2 Related Work

Sampling-based methods such as RRT and RRT* [7] have become standard in high-dimensional motion planning. Extensions incorporating optimality, dynamics, and risk awareness exist but often lack formal safety guarantees.

Potential field and PDE-based navigation [8] have long been studied for path planning. Harmonic potential fields, governed by Laplace’s equation, ensure smoothness and avoid local minima, yet they do not directly provide a safety margin measurable in the CBF framework.

Control Barrier Functions [1, 5] ensure set invariance in control systems but require a known differentiable safety function. Existing methods often define $h(x)$ as analytic distances to obstacles, which may be nondifferentiable in complex maps. Our approach bridges this gap by generating $h(x)$ through a Poisson PDE that respects boundary geometry.

2.1 Integration of CBFs in Sampling-based Planning

The integration of Control Barrier Functions with sampling-based planners often falls into two main categories: those focusing on *control synthesis* and those focusing on *path filtering*.

Dynamic and Robust CBF-RRT* (Control Synthesis) More advanced methods directly incorporate the system’s dynamics into the planning process. For example, the Robust CBF-RRT* approach [11] uses CBF conditions, often solved via Quadratic Programming (QP), at each time step along a sampled edge to find a dynamically safe control input u . This approach guarantees safety for kinodynamic systems and can be extended to Robust CBF (RCBF) to handle bounded uncertainties [11]. Similarly, methods like LQR-CBF-RRT* aim for optimality by integrating the Linear Quadratic Regulator (LQR) with CBF

constraints, sometimes simplifying the process by avoiding the continuous QP solving [10]. While powerful, these methods incur a high online computational cost due to the repeated synthesis of safe control (e.g., solving QPs or LQR) during tree expansion.

Geometric CBF-RRT* (Path Filtering) In contrast, our method, *Safe-CBF-RRT**, uses a computationally lighter approach. It employs a simplified, discrete CBF condition applied to the path segment geometry rather than synthesizing a dynamic control trajectory. This shifts the focus from finding an optimally safe control to generating an inherently safe reference path with minimal online computational overhead.

Contrast with Tracking-Based Safety Methods (FaSTrack): Another influential paradigm for provable safety is the *FaSTrack* framework [4], which uses level-set methods and Hamilton-Jacobi-Isaacs (HJI) equations to compute the aforementioned Safe-to-Track Set ($\mathcal{S}$). While FaSTrack offers strong, worst-case guarantees against bounded disturbances and execution error, **it decouples planning from safety assurance**. The path must first be planned (e.g., via RRT*), and then the safe-to-track set is checked for feasibility, often leading to computationally demanding solutions for high-dimensional systems. In contrast, *Safe-CBF-RRT** integrates the safety constraint derived from the smooth Poisson field directly into the sampling and rewiring steps. This allows the planner to efficiently *generate inherently safe paths*, avoiding the need for expensive high-dimensional HJI solutions characteristic of FaSTrack-type approaches.

3 Preliminaries

3.1 RRT* Algorithm

The RRT* algorithm is an incremental, sampling-based motion planner that asymptotically converges to the optimal feasible path. At each iteration, a random sample x_{rand} is drawn from the state space, and the tree is extended toward this sample from its nearest node x_{near} . Unlike RRT, RRT* performs two additional operations that guarantee asymptotic optimality. First, it selects a parent for the new node x_{new} by choosing, among all nodes within a ball of radius r_n , the one that minimizes the cost-to-come plus the local connection cost. Second, it "rewires" the tree by checking whether any neighbor can reduce its cost by adopting x_{new} as a new parent. As the number of samples n grows, the radius $r_n = \gamma(\frac{\log n}{n})^{\frac{1}{d}}$ shrinks, ensuring both probabilistic completeness and convergence to the optimal path cost. This makes RRT* suitable for high-dimensional motion planning problems where exact optimal planning is computationally intractable [6], [7]. Algorithm is summarized in Algorithm 1.

3.2 Control Barrier Functions

Control Barrier Functions (CBFs) provide a formalism for ensuring safety in control systems by enforcing forward invariance of a safe set. Given a dynamical

Algorithm 1 Classical RRT* algorithm

Require: State space $\mathcal{X}$, free space $\mathcal{X}_{\text{free}} \subseteq \mathcal{X}$, initial state x_{init} , number of iterations N

```

1:  $T \leftarrow (\{x_{\text{init}}\}, \emptyset)$ 
2: for  $n = 1$  to  $N$  do
3:    $x_{\text{rand}} \sim \text{Unif}(\mathcal{X}_{\text{free}})$ 
4:    $x_{\text{near}} \leftarrow \text{Nearest}(T, x_{\text{rand}})$ 
5:    $x_{\text{new}} \leftarrow \text{Steer}(x_{\text{near}}, x_{\text{rand}})$ 
6:   if  $\text{CollisionFree}(x_{\text{near}}, x_{\text{new}})$  then
7:      $\mathcal{N} \leftarrow \text{Neighbors}(T, x_{\text{new}}, r_n)$ 
8:      $x_{\text{parent}} \leftarrow \arg \min_{x \in \mathcal{N}} (c(x) + \text{Cost}(x, x_{\text{new}}))$ 
9:      $T.\text{AddNode}(x_{\text{new}})$ 
10:     $T.\text{AddEdge}(x_{\text{parent}}, x_{\text{new}})$ 
11:    for all  $x \in \mathcal{N}$  do
12:      if  $c(x_{\text{new}}) + \text{Cost}(x_{\text{new}}, x) < c(x)$  then
13:         $T.\text{Rewire}(x, x_{\text{new}})$ 
14:      end if
15:    end for
16:  end if
17: end for
18:
19: return  $T$ 

```

system

$$\dot{x} = f(x) + g(x)u, \quad (1)$$

where $x \in \mathbb{R}^n$ is the state, $u \in \mathbb{R}^m$ is the control input, and f, g are locally Lipschitz continuous functions, a set

$$\mathcal{C} = \{x \in \mathbb{R}^n : h(x) \geq 0\} \quad (2)$$

is defined as *safe* if there exists a continuously differentiable function $h : \mathbb{R}^n \rightarrow \mathbb{R}$ such that for all $x \in \mathcal{C}$,

$$\sup_{u \in U} [L_f h(x) + L_g h(x)u] \geq -\kappa h(x), \quad (3)$$

where $L_f h$ and $L_g h$ are the Lie derivatives of h along f and g , respectively, and $\kappa > 0$ is a tuning parameter - the smaller κ is the more conservative is condition. This condition ensures that the system's trajectories remain within the safe set $\mathcal{C}$ for all future times.

3.3 Poisson Safety Function Generation

Let $\Omega \subset \mathbb{R}^2$ denote the workspace with boundary $\partial\Omega$ and obstacles $\mathcal{O}_i$. We define a vector potential field $\mathbf{u} = [u_x, u_y]^T$ solving

$$\left\{ \begin{array}{ll} \nabla^2 \mathbf{u} = 0 & \text{in } \Omega \\ \mathbf{u} = c_i(\mathbf{x} - \mathbf{c}_i) & \text{on } \partial\mathcal{O}_i \end{array} \right\}. \quad (4)$$

A scalar field $h(x, y)$, termed the *Poisson Safety Function (PSF)*, is then obtained by solving

$$\left\{ \begin{array}{ll} \nabla^2 h = -\|\nabla \mathbf{u}\| & \text{in } \Omega \\ h|_{\partial\Omega} = 0 & \text{on } \partial\mathcal{O}_i \end{array} \right\}, \quad (5)$$

ensuring $h > 0$ in free space. Gradients $\nabla h = [\partial h / \partial x, \partial h / \partial y]$ are computed using finite differences.

4 Safe CBF-RRT* Algorithm

We introduce several key modifications to the classical RRT* algorithm in order to incorporate safety constraints derived from the Poisson-based Control Barrier Function. **The most important changes are:**

1. **Edges are validated using a CBF condition** computed from the Poisson Safety Function (PSF) and its spatial gradients.
2. **A constant nominal traversal speed is assumed** when moving along each candidate edge. This assumption is important: slower motion leads to larger feasible sets under the CBF condition, meaning that *even narrow passages may be safe at lower velocities*, while the same passages are considered unsafe at higher speeds.
3. **Edges violating the CBF constraint are rejected**, preventing unsafe tree expansion even if the geometric collision check passes.
4. **A safety-weighted cost** is introduced to prefer paths that stay in regions with higher PSF values (more safety margin).

Together, these changes turn the sampling process itself into a safety-aware procedure, rather than merely filtering an already planned path.

4.1 Ensuring Safety with Poisson CBFs

The Poisson Safety Function $h(x)$ and its spatial derivatives provide a smooth, differentiable representation of obstacle boundaries and safety gradients. In Safe CBF-RRT*, these quantities are used in three ways:

- **Discrete CBF validation of each candidate edge.** For a short edge between nodes a and b , we assume motion with constant velocity $\dot{x} = v(b - a)/|b - a|$. The CBF condition —, sampled at multiple points p along the edge, determines whether traversal is safe. This directly links feasibility to assumed speed.
- **Safety-aware cost weighting.** Each edge is assigned a cost depending on both its length and average safety margin $\bar{h}$.
- **Optional control-feasibility check.** For edges that satisfy discrete CBF conditions, PSF gradients can be used to verify whether a continuous control input exists to maintain safety along the smooth trajectory.

4.2 Trajectory Cost with Safety Weighting

To encourage the planner to use safer portions of the workspace, Safe CBF-RRT* modifies the classical RRT* cost function as:

$$J(a, b) = cL + (1 - c)\frac{1}{\bar{h}}, \quad (6)$$

where $L = |b - a|$ is the geometric length, and $\bar{h}$ is the average PSF value along the edge. Parameter $c \in [0, 1]$ determines the trade-off between path efficiency and safety.

4.3 Summary of Safe-CBF-RRT*

The overall algorithm follows the standard RRT* structure but rejects edges that fail the CBF feasibility condition (dependent on assumed traversal speed). This results in a tree that grows preferentially through regions of high safety margin while still preserving the asymptotic optimality guarantees of RRT* for the modified cost.

5 Results

This section presents simulation results comparing Safe-CBF-RRT* against standard RRT* in a 2D environment with randomly located obstacles. It provides diversity of paths, safety margins, and path lengths.

5.1 Simulation Setup

The workspace was a square randomly located obstacles. The grid resolution was 100×100 . Parameters used were $\text{stepSize} = 2$, $\kappa = 20$, and $N_{\text{iter}} = 2000$.

There were three scenarios with varying obstacle densities and configurations:

- **Scenario I:** Moderately spaced obstacles with few wide corridors.
- **Scenario II:** Tightly packed obstacles with narrow passages and one wide corridor.
- **Scenario III:** Few bigger obstacles with one good, wide corridor.

Each scenario was run 100 times for both algorithms - it means with $c = 1$ (length optimized only) and $c = 0$ (safety optimized only), recording path length, average safety metric $\bar{h}$, number of iterations, and rejection ratio to show how many potentially unsafe segments are rejected from tree. Scenarios with $c = 1$ are run with two velocities: $2 \frac{m}{s}$ and $0.5 \frac{m}{s}$ to show ability of Safe-CBF-RRT* to find safer paths when moving slower.

Additionally for both algorithm calibrations corresponding iterations were run with same random seed to show how both algorithms behave in exactly same conditions.

For simulation purposes, a simple unicycle kinematic model was assumed:

$$\begin{cases} \dot{x} = v \cos(\theta) \\ \dot{y} = v \sin(\theta) \\ \dot{\theta} = \omega \end{cases}. \quad (7)$$

5.2 Performance

Scenario I Scenario I represents an environment with moderately spaced obstacles and relatively wide free-space corridors. The PSF field (Fig. 1a) exhibits smooth gradients and well-defined safety basins around obstacles, making it a representative baseline configuration.

Standard RRT* tends to exploit narrow gaps, often producing shorter but riskier paths (Fig. 1b). This behavior is reflected in the low average safety value ($\bar{h} = 0.00057$). In contrast, Safe-CBF-RRT* systematically rejects unsafe edges (48.7% of samples), forcing the tree to expand through regions of higher PSF values (Fig. 1c). As a result, the mean path length increases from 118.57 to 147.86, while the mean safety metric improves by 16%.

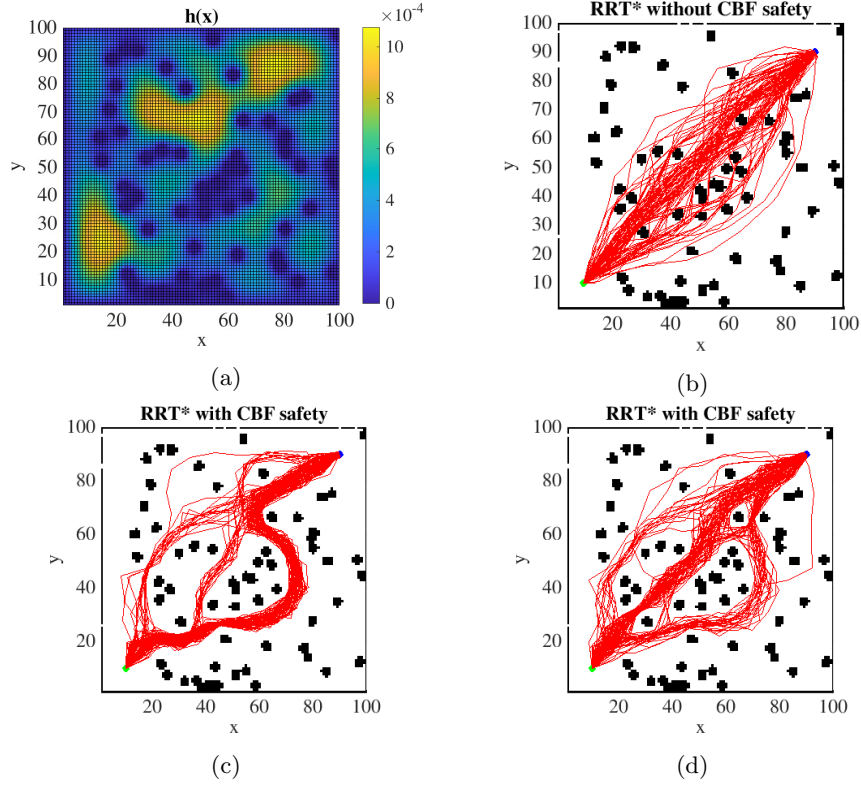


Fig. 1: Exemplary results of 100 different runs for Scenario I: (a) Control barrier function, (b) standard RRT*, (c) CBF-RRT* for $2 \frac{m}{s}$ target speed, (d) CBF-RRT* for $0.5 \frac{m}{s}$ target speed.

The higher number of iterations used by CBF-RRT* is consistent with the stronger filtering mechanism, which prioritizes safety over direct progress toward

the goal. Overall, Scenario I demonstrates that the proposed method effectively increases safety margins even in relatively easy environments.

For the reduced target speed of $0.5 \frac{m}{s}$, the impact of the CBF constraint becomes noticeably weaker. The rejection ratio decreases from 48.7% to 27.5%, and the required number of iterations drops accordingly (372). The resulting paths remain significantly safer than those generated by standard RRT* ($\bar{h} = 0.00065$), but their geometric length increases only moderately (128.49), remaining much closer to the baseline RRT* solution. This indicates that at lower target speeds the environment effectively becomes less restrictive, allowing Safe-CBF-RRT* to satisfy the safety constraints without forcing large detours. Consequently, the planner converges more efficiently while still benefiting from the PSF-based safety shaping.

Table 1: Performance comparison in Scenario I.

Algorithm	Length	$h(x)$	Iterations	Rejections
RRT*	118.57	0.00057	333	13.6%
CBF-RRT* ($2 \frac{m}{s}$)	147.86	0.00066	563	48.7%
CBF-RRT* ($0.5 \frac{m}{s}$)	128.49	0.00065	372	27.5%

Scenario II Scenario II introduces significantly tighter passages and higher obstacle density. The PSF field (Fig. 2a) shows steeper gradients and narrow channels of admissible motion, making safety enforcement more restrictive.

Standard RRT* frequently selects unsafe extensions in this environment, reflected by a much higher rejection ratio (29.0%) compared to Scenario I. Its solutions remain relatively short (119.84), but operate close to obstacles, maintaining only a moderate safety average ($\bar{h} = 0.00085$).

Safe-CBF-RRT* must navigate through a more constrained safety landscape, leading to an even larger rejection ratio (59.9%) and correspondingly more conservative motion. Although the resulting paths are longer (130.95), they traverse regions with noticeably higher PSF values, improving the average safety metric by 29% relative to standard RRT*.

Scenario II highlights the ability of the method to maintain safe operation under geometric constraints that naturally encourage risk-taking behavior in classical sampling-based planners.

When the target speed is reduced to $0.5 \frac{m}{s}$, the CBF filtering remains strong but becomes considerably less restrictive. The rejection ratio drops from 59.9% to 48.5%, and the number of iterations decreases from 537 to 407, indicating that the planner requires fewer alternative expansions to locate a safe homotopy class. The resulting trajectories exhibit nearly identical length (130.80) and safety level ($\bar{h} = 0.00109$) as in the $2 \frac{m}{s}$ case. This shows that even in highly constrained

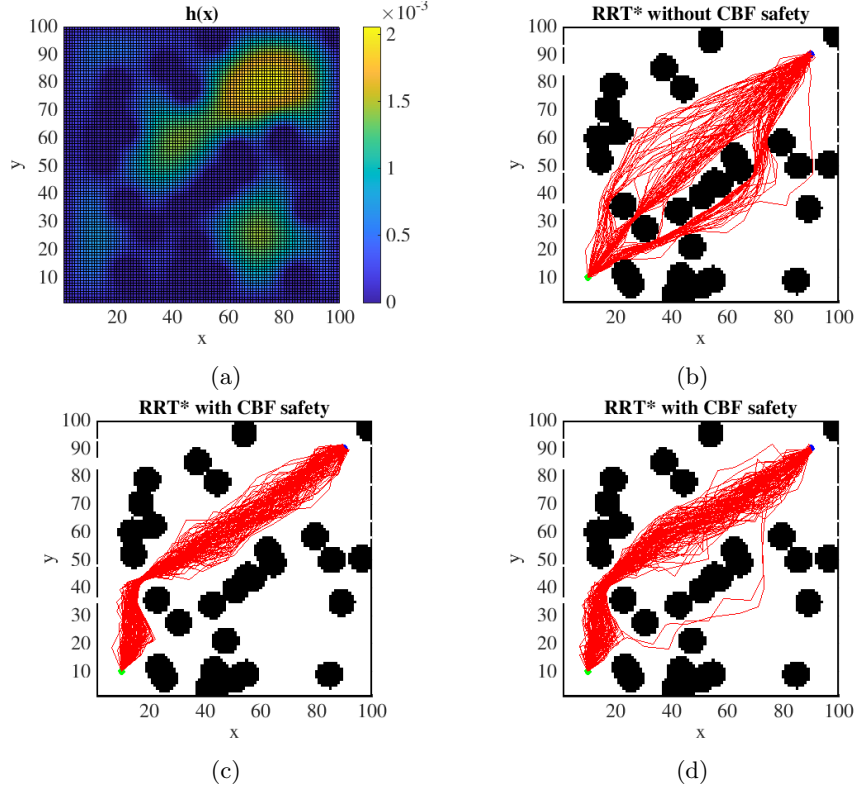


Fig. 2: Exemplary results of 100 different runs for Scenario II: (a) Control barrier function, (b) standard RRT*, (c) CBF-RRT* for $2\frac{m}{s}$ target speed, (d) CBF-RRT* for $0.5\frac{m}{s}$ target speed.

environments, lower forward velocity relaxes the safety constraint sufficiently to reduce computational effort while preserving the safety benefits of the proposed method.

Scenario III Scenario III consists primarily of an open environment with a few concentrated high-risk areas. The PSF distribution (Fig. 3a) features broad low-gradient regions interspersed with sharp drops near obstacles.

In this configuration, standard RRT* is able to find relatively direct paths (122.71), but due to the scattered obstacle layout, it often crosses regions of low safety potential. This is reflected in the lowest average safety value among all scenarios ($\bar{h} = 0.00037$).

Safe-CBF-RRT* improves the safety metric to $\bar{h} = 0.00048$ by rejecting 58.4% of unsafe expansions and steering the tree around low-PSF zones (Fig. 3c). Interestingly, the number of iterations is slightly lower than for standard RRT*, indicating that large open areas reduce the need for rewiring and costly explo-

Table 2: Performance comparison in Scenario II.

Algorithm	Length	$\bar{h}(x)$	Iterations	Rejections
RRT*	119.84	0.00085	345	29.0%
CBF-RRT* ($2\frac{m}{s}$)	130.95	0.00110	537	59.9%
CBF-RRT* ($0.5\frac{m}{s}$)	130.80	0.00109	407	48.5%

ration. Despite the safety constraints, the method only marginally increases the path length (from 122.71 to 126.32), demonstrating that when safe alternatives are abundant, the penalty for enforcing CBF constraints becomes minimal.

Scenario III shows that the proposed planner adapts gracefully to environments where safe path options are naturally available, avoiding unnecessary conservatism while still improving safety.

For the reduced speed of $0.5\frac{m}{s}$, the planner benefits from the abundance of safe alternatives: the rejection ratio decreases from 58.4% to 48.7%, and the number of iterations further drops to 362. Despite this, the safety level remains essentially unchanged ($\bar{h} = 0.00047$), and the resulting trajectory length (126.51) stays very close to the $2\frac{m}{s}$ variant. This confirms that in open environments the influence of the CBF constraint diminishes at lower speeds, since safe feasible expansions are readily available, and the planner does not require significant detours to maintain conservative operation.

Table 3: Performance comparison in Scenario III.

Algorithm	Length	$\bar{h}(x)$	Iterations	Rejections
RRT*	122.71	0.00037	421	39.8%
CBF-RRT* ($2\frac{m}{s}$)	126.32	0.00048	389	58.4%
CBF-RRT* ($0.5\frac{m}{s}$)	126.51	0.00047	362	48.7%

5.3 Discussion

The experimental evaluation across the three scenarios demonstrates that integrating a Poisson-based CBF into the RRT* framework substantially reshapes the planner’s behavior, consistently favoring safer regions of the workspace. The results reveal a clear trade-off between path optimality in terms of geometric length and improved safety, as measured by the Poisson Safety Function.

In Scenario I, where wide corridors dominate the environment, Safe-CBF-RRT* achieves a noticeable increase in safety margin at the cost of moderate path elongation. The rejection ratio of nearly 50% confirms that the PSF-based

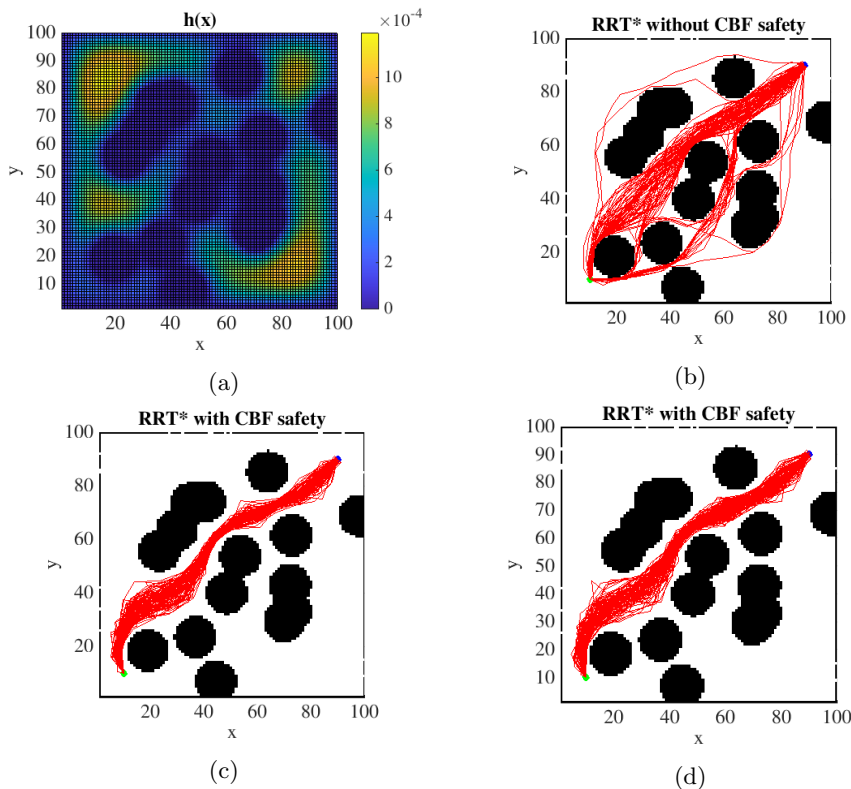


Fig. 3: Exemplary results of 100 different runs for Scenario III: (a) Control barrier function, (b) standard RRT*, (c) CBF-RRT* for $2 \frac{m}{s}$ target speed, (d) CBF-RRT* for $0.5 \frac{m}{s}$ target speed.

constraint actively filters out edges that steer too close to obstacles, even when those edges are geometrically efficient. Interestingly, despite higher rejection rates and longer exploration, the algorithm maintains stable convergence and achieves consistently safer trajectories across 100 runs.

In Scenario II, which contains narrow passages and densely packed obstacles, the effect of CBF filtering becomes even more pronounced. Standard RRT* often forces its way through the narrowest admissible gaps, leading to short but unsafe trajectories characterized by very low mean PSF values. The Safe-CBF-RRT* algorithm, in contrast, frequently avoids these bottlenecks entirely. As a result, the planner may favor longer detours through alternative corridors offering smoother PSF gradients. This scenario highlights the fundamental benefit of embedding CBF-based reasoning directly in the sampling process: unsafe topological options are pruned early, and the planner adapts by allocating more iterations to exploring safer homotopy classes. Consequently, Safe-CBF-RRT*

exhibits both higher path length variance and significantly elevated $\bar{h}$, which is aligned with the intended conservative behaviour.

In Scenario III, where the environment is sparse and open, the difference between the two planners becomes less dramatic. With obstacles exerting weaker influence on the PSF gradients, most edges considered by RRT* are already safely admissible. The rejection ratio drops, and the resulting paths of both algorithms approach similar lengths. Nevertheless, Safe-CBF-RRT* still yields slightly higher average safety values, indicating that the PSF weighting in the cost function successfully biases the rewiring step toward more robust trajectories even when explicit filtering plays a minor role. This scenario illustrates that the method does not degrade performance in trivially safe environments, maintaining comparable efficiency while providing a marginal safety improvement at virtually no additional planning cost.

Across all scenarios, the experiments with identical random seeds show that the algorithms diverge early in the exploration stage once the CBF constraint begins rejecting unsafe edges. This confirms that the proposed method influences the global structure of the growing tree rather than acting as a lightweight post-filter. The mean number of iterations also consistently increases for Safe-CBF-RRT*, reflecting the planner’s need to explore alternative safe regions when the straightforward geometric extension is disallowed.

Finally, the comparison between the $c = 1$ (pure length optimization) and $c = 0$ (pure risk minimization) regimes indicates that the PSF-based safety metric interacts naturally with the RRT* rewiring mechanism. For $c = 0$, the planner strongly prefers wide, smooth-value PSF basins, sometimes yielding significantly longer but uniformly safe paths. For intermediate values of c , the trade-off can be tuned continuously depending on application needs (robot-human interaction, autonomous navigation in clutter, or low-risk operation for high-cost platforms).

Overall, the results confirm that Poisson-based CBF integration provides a principled mechanism for steering the sampling process toward inherently safe trajectories. It increases robustness in complex environments while preserving the asymptotic optimality structure of RRT*. The approach is especially advantageous in environments with narrow or ambiguous safety gradients, where purely geometric sampling is insufficient to avoid risky configurations.

6 Conclusion and Future Work

We presented *Safe-CBF-RRT**, a motion planning framework combining RRT* with a Control Barrier Function derived from a Poisson Safety Field. The approach unifies PDE-based potential field generation with formal safety guarantees **in a manner computationally lighter than tracking-based methods (e.g., FaSTrack)**, while actively shaping the path for safety. Future work will extend this method to kinodynamic models, uncertain environments, and real robotic experiments.

References

1. A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada. Control barrier functions: Theory and applications. In *2019 18th European Control Conference (ECC)*, pages 3420–3431, 2019.
2. G. Bahati and A. D. Ames. Safe set synthesis with tunable boundary gradients via poisson safety functions.
3. G. Bahati, R. M. Bena, and A. D. Ames. Dynamic safety in complex environments: Synthesizing safety filters with poisson’s equation, 2025.
4. M. Chen, S. L. Herbert, H. Hu, Y. Pu, J. F. Fisac, S. Bansal, S. Han, and C. J. Tomlin. Fastrack: a modular framework for real-time motion planning and guaranteed safe tracking. *IEEE Transactions on Automatic Control*, 66(12):5861–5876, Dec. 2021.
5. K. Garg, J. Usevitch, J. Breeden, M. Black, D. Agrawal, H. Parwana, and D. Panagou. Advances in the theory of control barrier functions: Addressing practical challenges in safe control synthesis for autonomous and robotic systems, 2023.
6. S. Karaman and E. Frazzoli. Incremental sampling-based algorithms for optimal motion planning, 2010.
7. S. Karaman and E. Frazzoli. Sampling-based algorithms for optimal motion planning, 2011.
8. G. Klančar, A. Zdešar, and M. Krishnan. Robot navigation based on potential field and gradient obtained by bilinear interpolation and a grid-based search. *Sensors*, 22(9), 2022.
9. S. M. LaValle. Rapidly-exploring random trees : a new tool for path planning. *The annual research report*, 1998.
10. G. Yang, M. Cai, A. Ahmad, A. Prorok, R. Tron, and C. Belta. Lqr-cbf-rrt*: Safe and optimal motion planning, 2023.
11. G. Yang, B. Vang, Z. Serlin, C. Belta, and R. Tron. Sampling-based motion planning via control barrier functions. In *Proceedings of the 2019 3rd International Conference on Automation, Control and Robots*, ICACR 2019, page 22–29. ACM, Oct. 2019.